NEXUS-R Full Phase 2 Context

> Complete system context for GPT, Claude, DeepSeek, or any AI coding agent. Paste this entire document at the start of your prompt. Date: 2026-05-24 Version: Phase 2, 90% complete (v0.3.0-in-progress)

SECTION 0: CRITICAL RULES FOR AGENTS

1. **NEVER modify specs/** — Specs are immutable contracts. If you find a bug, report it. Do not fix it.
2. **Work in isolation** — Only read your module's `interface.yaml` and foundation code. Do not read other modules' implementations.
3. **Write tests first** — Every change must have adversarial and property-based tests.
4. **Verification precedes integration** — No module enters without passing tests a second agent validates.
5. **Event-source everything** — Log every decision, test result, integration outcome.
6. **No AGI hype** — This is infrastructure, not intelligence. Be honest about limitations.
7. **Stop on ambiguity** — If a spec is unclear, ask. Do not guess.

SECTION 1: PROJECT OVERVIEW

NEXUS-R is a personal agent runtime that makes AI task execution progressively cheaper through verified workflow reuse.

Core thesis: Every successful execution makes the next one faster and cheaper — not through subscription discounts, but through a verified cache of reusable execution plans called Execution Trace Distillation (ETD).

Current status: Phase 2, 90% complete. 222 tests passing. ETD proven with 100% cost reduction on repeated tasks.

Repository: `C:\Users\Gaurav\Documents\NEXUS-R\nexus-r`

SECTION 2: ARCHITECTURE (5 SUBSYSTEMS, 6 MODULES)

Subsystem Map

```
Input Gateway (Intent Parser) → Cognition Router (CAR + ISE)
         ↓
Trust Layer (cross-cutting: permissions, audit, cost, secrets)
         ↓
Execution Sandbox (MCP tools, inline verification)
         ↓
State Core (Event Store, Working State, Identity)
         ↓
Workflow Engine (ETD pipeline)
```

Module Directory Structure

```
nexus-r/
├── foundation/nexus_r/
│   ├── events.py          # Event, CausalEvent, EventStore (SQLite, append-only)
│   ├── models.py          # ModelRouter (LiteLLM wrapper), ProviderProfile
│   ├── config.py          # NEXUSConfig (Pydantic settings)
│   ├── exceptions.py      # NEXUSError hierarchy
│   └── telemetry.py       # TelemetrySpan, structured JSON logging
│
├── modules/
│   ├── input_gateway/     # Intent parsing, classification, parameter extraction
│   ├── cognition_router/  # CAR, parallel probe, de-escalation, capability profiler
│   ├── execution_sandbox/ # MCP client, sandbox, verifier, permission checker
│   ├── state_core/        # EventStore, WorkingState, IdentityStore, projected views
│   ├── workflow_engine/   # ETD pipeline (7 stages), applicator, pipeline orchestrator
│   ├── trust_layer/       # Permission enforcer, risk classifier, prompt injection defense
│   ├── session_manager/   # Session lifecycle, recovery, stale pointer repair
│   └── cli/               # Typer CLI: nexus run, history, cost, config
│
├── specs/                 # SOURCE OF TRUTH. Read, never write.
│   ├── 00_architecture.md
│   ├── 01_input_gateway.spec.md
│   ├── 02_cognition_router.spec.md
│   ├── 03_execution_sandbox.spec.md
│   ├── 04_state_core.spec.md
│   ├── 05_workflow_engine.spec.md
│   ├── 06_trust_layer.spec.md
│   ├── 07_cost_dashboard.spec.md
│   └── 99_acceptance_criteria.md
│
├── tests/
│   ├── unit/              # Per-module tests (target: 90% coverage)
│   ├── integration/       # Cross-module pipeline tests
│   ├── security/          # Adversarial and hardening tests
│   ├── stress/            # Concurrency, memory, load tests
│   └── failure/           # Chaos engineering, crash recovery
│
├── scripts/
│   ├── phase1_benchmark.py
│   ├── phase1_byok_validation.py
│   ├── phase1_failure_injection.py
│   ├── phase1_interruption_validation.py
│   ├── phase1_real_validation.py
│   ├── etd_cost_reduction_test.py    # KILL CRITERION TEST
│   └── long_running_validation.py
│
└── docs/
    ├── architecture_review_phase1.md
    ├── benchmark_report_phase1.md
    ├── security_audit_phase1.md
```

```
├── known_limitations_phase1.md
├── technical_debt_phase1.md
├── runtime_truthfulness_report_phase1_5.md
├── session-runtime-failure-report.md
├── persistence-redesign-proposal.md
├── recovery-architecture-proposal.md
├── concurrency-risk-report.md
└── final_operational_assessment_phase2_ready.md
```

SECTION 3: KEY DESIGN DECISIONS

Decision	Rationale	Status
Event-sourced everything	Enables replay, debugging, ETD extraction	✅
MCP everywhere	Reduces integration surface, ensures interoperability	✅
LiteLLM for model abstraction	Don' t build provider router, use proven library	✅
Deny-first security	Default deny; allow is earned	✅
Privacy as hard constraint	Local models always preferred when flag set	✅
CPC short-circuit before routing	Check cache first —primary cost-saving mechanism	✅
Explicit generalization bounds	No workflow without >90% proven success	✅
Inline verification	Catches errors at point of occurrence	✅

SECTION 4: ETD PIPELINE (CORE INNOVATION)

What ETD Is

Execution Trace Distillation is a **verified cache of reusable execution plans** derived from successful traces. It is NOT "cognitive compression" and NOT "memory." It stores proven execution strategies with quantified reliability.

Seven-Stage Pipeline

Stage	Class	File	Input	Output	Key Method
1. Trace Recording	TraceRecorder	trace_recorder.py	Execution steps	list[CausalEvent]	record_step()
2. Distillation	TraceDistiller	distiller.py	list[CausalEvent]	DistilledWorkflow	distill()
3. Generalization	GeneralizationVerifier	generalizer.py	DistilledWorkflow + variants	GeneralizationResult	verify()

Table 2 – continued

Stage	Class	File	Input	Output	Key Method
4. Parameterization	WorkflowParameterizer	parameterizer.py	DistilledWorkflow	ETDEntry	parameterize()
5. Indexing	ETDIndexer	indexer.py	ETDEntry	IndexedETDEntry	index()
6. Retrieval	ETDRetriever	retriever.py	IntentResult	RankedETDEntry	retrieve()
7. Invalidation	ETDInvalidator	invalidator.py	IndexedETDEntry	ETDInvalidationRule	run_all()
Apply	ETDApplicator	applicator.py	ETDEntry + params	ExecutionResult	apply()
Orchestrate	ETDPipeline	pipeline.py	Various	Various	process_success(), find_match()

ETD Entry Data Structure

```python
@dataclass
class ETDEntry:
    id: str                          # "etd_" + hash
    intent_signature: str            # e.g., "deploy-nextjs-to-vercel"
    intent_embedding: list[float]    # 128-dim hash vector
    normalized_input: str            # "deploy my nextjs app to vercel"
    input_schema: dict               # {"project_dir": "string", "env_vars": "map"}
    output_schema: dict              # {"deploy_url": "string", "status": "enum"}
    tool_sequence: list[ToolStep]    # Ordered essential steps
    parameter_slots: list[str]       # ["project_dir", "env_vars", "deploy_url"]
    invariant_checks: list[str]      # ["node >= 18", "vercel-cli installed"]
    success_count: int               # Incremented on success
    failure_count: int               # Incremented on failure
    generalization_success_rate: float # 0.0 to 1.0, must be >= 0.90
    last_validated: str              # ISO timestamp
    avg_cost: float                  # Average historical cost
    avg_latency_ms: float            # Average historical latency
```

Critical Implementation Detail

Embedding must use `normalized_input` (not `intent_signature`):

```python
# CORRECT — both query and storage use same string
query_embedding = compute_embedding("list all python files")
stored_embedding = compute_embedding("list all python files")  # from normalized_input
# Similarity ≈ 1.0 → ETD match

# WRONG — different strings → different hashes
query_embedding = compute_embedding("list all python files")
stored_embedding = compute_embedding("list-files")  # from intent_signature
# Similarity ≈ 0 → NO MATCH → ETD never works
```

This bug was found during validation and fixed. Always use `normalized_input` for embeddings.

Kill Criterion Results

Metric	Result	Target	Status
Cost reduction	**100%**	40%	✅ PASS
Latency reduction	**48.5%**	40%	✅ PASS
ETD hit rate	**80%** (4/5)	—	✅ Strong

Bugs Found and Fixed

Bug	Root Cause	Fix
Embedding mismatch	Query from `normalized_input`, stored from `intent_signature`	Recompute embedding from `normalized_input`
Type-match delimiter	`list_files` vs `list-files`	Normalize both to hyphen
Applicator action mapping	`execution_sandbox` mapped to wrong action type	Map to `None`, use action string directly

---

SECTION 5: COGNITION ROUTER (CAR)

6-Tier Fallback Chain

```
local_7b → local_14b → local_70b → byok_budget → byok_frontier → managed_premium
```

Rules:

- Never jump more than 1 tier at a time
- Cost cap enforced at each tier
- `managed_premium` always requires explicit user approval

Parallel Probe

```
# Send to T1 + T2 simultaneously
# If T1 succeeds verification → discard T2 result
# Return whichever succeeds first
probe_result = await parallel_probe.probe(task, tier1="local_7b", tier2="local_14b")
```

De-escalation Learning

```
# After 5 consecutive successes at lower tier than classified:
# Suggest downgrade for similar tasks
# Persist in IdentityStore
if consecutive_successes >= 5 and actual_tier < assigned_tier:
    de_escalation.suggest_downgrade(task_signature, actual_tier)
```

SECTION 6: TRUST LAYER

Permission Tiers

Tier	Actions	Approval
T1	Read files, read-only terminal, web search, safe MCP	Auto
T2	Write workspace files, sandboxed terminal, HTTP GET approved	Auto + log
T3	Arbitrary terminal, external files, HTTP POST/PUT/DELETE, browser, API keys	**User prompt**
T4	Delete files, access secrets, deploy to prod, financial transactions	**Explicit confirm + reason**

Risk Classifier Rules (R1-R10)

```
R1: destructive ops (rm, del) → DENY
R2: risky ops (sudo, chmod 777) → REVIEW
R3: routine ops (ls, cat, echo) → PASS
R4: external network (curl to unknown) → REVIEW
R5: mass deletion (>10 files) → DENY
R6: system directory access → DENY
R7: repeated failures (5+ in 1h) → REVIEW
R8: unknown tool invocation → REVIEW
R9: large file write (>100MB) → REVIEW
R10: secret in command → DENY
```

Performance: <10ms per 100 classifications.

Prompt Injection Defense

Pattern	Detection Rate
ignore_instructions	91.2%
role_escalation	91.2%
secret_extraction	91.2%
command_injection	91.2%
prompt_leak	91.2%
delimiter_breakout	91.2%

False positive rate: <15%.

SECTION 7: STATE CORE

Three-Tier Memory

Tier	Name	Stores	Lifecycle	Storage
1	Volatile	Working State — current task context	Dies when task completes	In-memory dict
2	Durable	Event Store — immutable log of all actions	Append-only, 90 days	SQLite + WAL
3	Identity	User Profile — preferences, keys, policies	Slow-changing, encrypted	Local encrypted file

EventStore Performance

Metric	Result
Batched append (10K)	0.024 ms/event
Single append	1.16 ms/event
Read by ID	0.15 ms
Causal chain (100 events)	12.5 ms

Session Recovery

- SQLite session catalog for session rows, rollout metadata, active pointers
- Immutable JSON snapshots: temp-file + `os.replace` + `fsync`
- Canonical workspace-path validation
- Stale-pointer repair
- **Cross-store atomicity: best-effort** (known limitation)

SECTION 8: EXECUTION SANDBOX

Sandbox Layers

Layer	Implementation	Scope
Terminal	Docker container or subprocess with restrictions	Workspace only
Filesystem	Path scoping, parent traversal blocked	Workspace directory
Browser (Phase 3)	Playwright + isolated Chromium	No host filesystem
API calls	Domain whitelisting, credential injection	Approved domains only

Inline Verification

Every step verified after execution:

- Terminal: `exit_code == 0`
- Filesystem: `file_exists`, `content_matches`
- API: `status == 200`
- Browser: `element_found`

SECTION 9: PROVIDER STACK

Local (Primary)

Model	Params	VRAM	Cost	Status
Qwen3-Coder-Next	80B total, 3B active	16GB	$0	✅ Validated

BYOK (Fallback)

Provider	Model	Cost	Status
Groq	llama-3.3-70b-versatile	~$0.001/1K	✅ Validated
NVIDIA NIM	qwen3-coder-480b-a35b	$0 (free tier)	⏳ Wired, not validated
Anthropic	claude-sonnet-4.1	~$3/1K	⏳ Not configured

SECTION 10: TESTING STANDARDS

Coverage Targets

Module	Minimum	Target
Foundation	80%	90%
Input Gateway	80%	90%
Cognition Router	80%	90%
Execution Sandbox	80%	90%
State Core	80%	90%
Workflow Engine	80%	90%
Trust Layer	80%	90%

Test Categories

Category	Purpose	Location
Unit	Module correctness in isolation	`tests/unit/test_{module}.py`
Adversarial	Injection, traversal, spoofing	`tests/unit/test_{module}_adversarial.py`
Integration	Cross-module pipeline	`tests/integration/test_*.py`
Security	Vulnerability hardening	`tests/security/test_*_hardening.py`
Stress	Concurrency, memory, load	`tests/stress/test_*.py`
Failure	Chaos engineering	`tests/failure/test_*.py`

Running Tests

```
# All tests
pytest tests/ -v

# With coverage
pytest --cov=nexus_r --cov-report=term-missing

# Specific module
pytest tests/unit/test_distiller.py -v

# Integration
pytest tests/integration/test_etd_pipeline.py -v

# Kill criterion
python scripts/etd_cost_reduction_test.py
```

SECTION 11: ERROR CODE RANGES

Module	Range	Example
Input Gateway	IG-001 to IG-050	IG-014: Invalid intent classification
Cognition Router	CR-001 to CR-100	CR-031: Cost cap exceeded
Execution Sandbox	ES-001 to ES-080	ES-045: Sandbox escape detected
State Core	SC-001 to SC-050	SC-022: Event chain broken
Workflow Engine	WE-001 to WE-100	WE-067: Generalization below threshold
Trust Layer	TL-001 to TL-100	TL-089: T4 approval denied
Session Manager	SM-001 to SM-050	SM-012: Session expired
Cost Dashboard	CD-001 to CD-050	CD-023: WebSocket connection failed
Foundation	NX-001 to NX-050	NX-001: Configuration invalid

SECTION 12: NAMING CONVENTIONS

Files

```
# Correct
modules/workflow_engine/src/distiller.py
modules/cognition_router/src/parallel_probe.py

# Incorrect
modules/workflow/src/distill.py  # missing subsystem clarity
modules/router/src/parallel.py   # too generic
```

Classes

```
# Correct
```

```
class TraceDistiller:
class GeneralizationVerifier:
class ETDPipeline:

# Incorrect
class Distiller:           # too generic
class IVerifier:           # Hungarian notation
class PipelineImpl:        # implementation suffix
```

Functions

```
# Correct
def distill(trace: list[CausalEvent]) -> DistilledWorkflow:
def verify(workflow: DistilledWorkflow, variants: list[TaskVariant]) -> GeneralizationResult:
def classify_risk(action: Action, target: str, scope: str) -> RiskScore:

# Incorrect
def process():             # too generic
def do_distill():          # weak verb
def handle_stuff():        # vague
```

SECTION 13: CURRENT WORK IN PROGRESS

Phase 2 Remaining

Feature	Spec File	Prompt	Status
Cost dashboard (FastAPI)	`07_cost_dashboard.spec.md`	18	⏳ Building

Phase 2 Complete

Feature	Status
ETD pipeline (7 stages)	✅
6-tier CAR	✅
Parallel probe	✅
De-escalation learning	✅
T3-T4 permissions	✅
Risk classifier	✅
Prompt injection defense	✅

Phase 3 Planned

Feature	Status
Browser automation (Playwright)	⏳ Spec’ d
ETD composition (chaining)	⏳ Spec’ d

Table 17 – continued

Feature	Status
Three-tier memory (vector embeddings)	⏳ Spec' d
Execution replay and fork	⏳ Spec' d
Polished Web UI	⏳ Spec' d
Session management for authenticated websites	⏳ Spec' d

Phase 4 Planned

Feature	Status
ETD community exchange	⏳ Spec' d
Advanced CAR (learned classifier)	⏳ Spec' d
Mobile companion	⏳ Spec' d
Third-party API	⏳ Spec' d

SECTION 14: KNOWN LIMITATIONS

Limitation	Severity	Mitigation
Cross-store atomicity (event + session)	Medium	Best-effort, documented
IdentityStore corruption recovery	Medium	No typed recovery path
ETD latency ceiling (3.7s cached)	Low	Sandbox overhead, not ETD logic
BYOK frontier unvalidated	Low	NVIDIA key not tested
30-min stability soak not run	Low	Bounded memory, documented
Browser automation not built	Medium	Phase 3 feature
Cost dashboard not built	Medium	Prompt 18 in progress
In-memory ETD store	Medium	Not persistent across restarts
No database migrations	Low	Schema changes risky
Windows path edge cases	Low	Fresh UX tested

SECTION 15: HOW TO WORK WITH THIS CODEBASE

For New Contributors

1. Read `specs/00_architecture.md`
2. Read `CONTRIBUTING.md`
3. Set up environment: `pip install -e ".[dev]"`
4. Pull Ollama model: `ollama pull qwen3-coder-next:q4_K_M`
5. Run demo: `nexus run "hello"`
6. Run tests: `pytest tests/unit/ -q`

For Agent Contributors

1. Read the relevant `.spec.md` file
2. Do NOT modify specs
3. Work in isolation — only your module + foundation
4. Write tests first (adversarial + property-based)
5. Pass security audit before integration
6. Log every decision

For Meta-Agent (Human Operator)

1. Route tasks to appropriate agent role
2. Feed context — specs + foundation + module interface
3. Apply patches — review agent output, merge if correct
4. Verify — run tests, check coverage, validate behavior
5. Log — commit with clear messages, update reports

SECTION 16: CRITICAL CONTEXT FOR NEXT PROMPT

If you are an AI agent reading this to implement the next feature, here is what you MUST know:

If Implementing Cost Dashboard (Prompt 18)

Context:

- State Core provides projected views for billing and audit data
- EventStore has cost events with task_id, model_used, tokens, cost
- ETDStore has cached workflows with success rates
- CostTracker emits events that should push to WebSocket

Files to read:

- `specs/07_cost_dashboard.spec.md`
- `modules/state_core/src/projected_views.py`
- `modules/trust_layer/src/cost_tracker.py`
- `foundation/nexus_r/events.py`

Files to create:

- `modules/web_ui/src/app.py` (FastAPI)
- `modules/web_ui/src/static/index.html`
- `modules/web_ui/src/static/app.js`
- `tests/integration/test_cost_dashboard.py`

Acceptance:

- Dashboard displays correct cost data from EventStore
- Audit log is searchable and paginated

- ETD list shows all cached workflows with stats
- Real-time cost updates work via WebSocket
- UI is responsive and functional
- Coverage >= 90%

If Implementing Phase 3 Features

Wait. Phase 2 must be 100% complete (cost dashboard built and tested) before starting Phase 3. The spec files for Phase 3 exist but are not yet activated.

If Fixing Bugs

Rules:

1. Write failing test first
2. Fix the bug
3. Verify test passes
4. Run full test suite: `pytest tests/`
5. Update `known_limitations_phase1.md` if limitation removed
6. Commit: `fix(module): description`

SECTION 17: CONTACT AND REPORTS

Report	Location
Architecture review	`docs/architecture_review_phase1.md`
Benchmarks	`docs/benchmark_report_phase1.md`
Security audit	`docs/security_audit_phase1.md`
Known limitations	`docs/known_limitations_phase1.md`
Technical debt	`docs/technical_debt_phase1.md`
Runtime truthfulness	`docs/runtime_truthfulness_report_phase1_5.md`
Operational assessment	`docs/final_operational_assessment_phase2_ready.md`

> This document is the single source of truth for NEXUS-R Phase 2. If you are an AI agent, read this entire document before writing any code. If something is unclear, ask. Do not guess. Specification is the product. Verification precedes integration.